

計算機ゼミ 課題 4

石川曹太

2026 年 7 月 10 日

1 問題設定

今回の課題では、一次元拡散方程式を数值的に解き、その結果を解析解と比較することで数値解法の精度を検証する。拡散方程式は熱伝導現象や物質拡散など、時間とともに物理量が空間的に広がっていく現象を表す代表的な偏微分方程式であり、数値解析の基礎として広く用いられている。

今回の課題で対象とする方程式は、以下で与えられる。

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial x^2} \quad (1)$$

ここで、 $u(x, t)$ は位置 x 、時刻 t における未知関数であり、熱伝導問題では温度、拡散問題では濃度に対応する。また、 ν は拡散係数と呼ばれ、拡散のしやすさを表す定数である。本課題において初期条件は、

$$u(x, 0) = \begin{cases} 0 & (x = 0) \\ 1 & (\text{otherwise}) \end{cases} \quad (2)$$

となる。また、境界条件は左端と右端で異なり、それぞれ

$$u(0, t) = 0, \quad \partial_x u(L, t) = 0 \quad (3)$$

である。一つ目の式は Dirichlet 境界条件と呼ばれ、境界での未知量そのものを既知値として固定する条件である。一方二つ目の式は Neumann 境界条件と呼ばれ、境界における空間微分を指定する条件であり、本課題では右端における流束が存在しないことを意味している。

2 格子点の導入

偏微分方程式をコンピュータで数值的に解くためには、連続的な空間と時間を有限個の点へ分割する必要がある。そのため、空間方向および時間方向をそれぞれ離散化し、各格子点における未知量を計算する。

まず初めに解析領域 $0 \leq x \leq L$ を N_x 等分し、空間の刻み幅を

$$\Delta x = \frac{L}{N_x} \quad (4)$$

と定義する．この時の各格子点の座標は，

$$x_i = i\Delta x \quad (i = 0, 1, \dots, N_x) \quad (5)$$

で表される．

また，時間方向についても一定の時間の刻み幅 Δt を用いて離散化し，以下のように定義する．

$$t^n = n\Delta t \quad (n = 0, 1, 2, \dots) \quad (6)$$

これらの格子点における未知関数は

$$u_i^n = u(x_i, t^n) \quad (7)$$

と表すことにする．ここで，添え字 i は空間方向の格子番号，添え字 n は時間方向の格子番号を表している．このように連続な関数を有限個の格子点上の値で表現することにより，偏微分方程式を代数方程式へ変換し，コンピュータによる数値計算が可能となる．

3 陽解法

陽解法とは，現在時刻における未知量のみを用いて次の時刻の未知量を計算する手法である．そのため，一度現在時刻の値が求まれば，行列計算などを行うことなく順次時間発展させることができる．この特徴から計算手順が単純であり，実装が容易であるという利点を持つ．

陽解法では式 (1) における左辺 (時間項) を現在時刻 t^n と次の時刻の t^{n+1} の値を用いた前進差分によって近似する．

$$\frac{\partial u}{\partial t} \approx \frac{u_i^{n+1} - u_i^n}{\Delta t} \quad (8)$$

一方，式 (1) における右辺 (空間方向の二階微分) は中央差分によって以下のように近似可能である．

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} \quad (9)$$

ここで，右辺の空間微分は全て現在時刻 n における値で評価していることに注意する．このように現在時刻の情報のみを利用することが，陽解法と呼ばれる理由である．これらを支配方程式 (1) に代入すると，

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \nu \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} \quad (10)$$

を得る．ここで両辺に Δt をかけて未知量 u_i^{n+1} について整理すると，以下の式を得る．

$$u_i^{n+1} = u_i^n + r(u_{i+1}^n - 2u_i^n + u_{i-1}^n), \quad r = \nu \frac{\Delta t}{\Delta x^2} \quad (11)$$

ここで r は離散化パラメータである。この式 (11) が本課題で Fortran プログラムに実装した陽解法の更新式である。実際のプログラムでは、この式を内部格子点 $i = 1, \dots, N_x - 1$ に対して繰り返し適用することで、時刻 $t = 0$ から最終時刻 T_{max} まで数値解を逐次計算した。

陽解法は計算手順が単純で実装も容易である一方、時間刻み幅を自由に設定できるわけではない。時間刻み幅が大きすぎると、数値誤差が時間とともに増幅し、数値解が発散する可能背うがある。そこで、一次元拡散方程式に対する陽解法では、Von Neumann 安定性解析より、離散化パラメータ r は

$$r = \nu \frac{\Delta t}{\Delta x^2} \leq 0.5 \quad (12)$$

を満たさなければならないことが知られている。そのため、本課題で作成したプログラムでは、計算開始時に離散化パラメータを計算し、この条件を満たしているか確認した。条件を満たさない場合には数値計算を実行せず、プログラムを終了するように実装している。

4 陰解法

陰解法では、時間方向の差分近似は陽解法と同様に前進差分を用いるが、空間方向の二階微分を次の時刻 $n + 1$ における未知量で評価する点が陽解法との大きな違いである。

時間方向の微分は陽解法と同様に以下のように近似する。

$$\frac{\partial u}{\partial t} \approx \frac{u_i^{n+1} - u_i^n}{\Delta t} \quad (13)$$

一方、空間方向の二階微分に関しては、現在時刻ではなく次の時刻の値を用いて以下のように近似する。

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{\Delta x^2} \quad (14)$$

これらの式を拡散方程式 (1) に代入すると、

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \nu \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{\Delta x^2} \quad (15)$$

を得る。ここで両辺に Δt をかけて u_i^n について整理すると、以下の式を得る。

$$-ru_{i-1}^{n+1} + (1 + 2r)u_i^{n+1} - ru_{i+1}^{n+1} = u_i^n \quad (16)$$

これが陰解法における基本的な差分方程式である。

式 (16) を見ると、ある格子点 i の未知量だけではなく u_{i-1}^{n+1} , u_i^{n+1} , u_{i+1}^{n+1} の 3 つが同時に含まれていることが分かる。つまり、ある 1 点だけを計算することができず、すべての内部格子点を同時に求めなければならない

ない．そこで，各格子点の式をまとめて一つの連立一次方程式として表現する．内部節点のみを未知数として並べたベクトルを

$$\mathbf{u}^{n+1} = \begin{bmatrix} u_1^{n+1} \\ u_2^{n+1} \\ \vdots \\ u_{N_x-1}^{n+1} \end{bmatrix}$$

と定義する．同様に，現在時刻における未知量ベクトルを

$$\mathbf{u}^n = \begin{bmatrix} u_1^n \\ u_2^n \\ \vdots \\ u_{N_x-1}^n \end{bmatrix}$$

と定義する．式 (16) をすべての内部節点について書き並べると，

$$\begin{aligned} (1+2r)u_1^{n+1} - ru_2^{n+1} &= u_1^n, \\ -ru_1^{n+1} + (1+2r)u_2^{n+1} - ru_3^{n+1} &= u_2^n, \\ -ru_2^{n+1} + (1+2r)u_3^{n+1} - ru_4^{n+1} &= u_3^n, \\ &\vdots \\ -ru_{N_x-2}^{n+1} + (1+r)u_{N_x-1}^{n+1} &= u_{N_x-1}^n. \end{aligned}$$

これらの連立方程式を行列形式で表すと，

$$\begin{bmatrix} 1+2r & -r & 0 & \cdots & 0 \\ -r & 1+2r & -r & \ddots & \vdots \\ 0 & -r & 1+2r & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -r \\ 0 & \cdots & 0 & -r & 1+r \end{bmatrix} \begin{bmatrix} u_1^{n+1} \\ u_2^{n+1} \\ \vdots \\ u_{N_x-2}^{n+1} \\ u_{N_x-1}^{n+1} \end{bmatrix} = \begin{bmatrix} u_1^n \\ u_2^n \\ \vdots \\ u_{N_x-2}^n \\ u_{N_x-1}^n \end{bmatrix} \quad (28)$$

となる．本課題におけるプログラムでは，この係数行列を $\mathbf{A}$ とし，課題 3 のモジュールを用いて逆行列を計算することで，現在の時刻におけるすべての内部点から次の時刻におけるすべての内部点における数値を求める式を以下のように得る．

$$\mathbf{u}^{n+1} = \mathbf{A}^{-1}\mathbf{u}^n \quad (17)$$

ここで用いた陰解法という手法は，各時間ステップで連立一次方程式を解く必要があるため計算量は増加するものの，一次元拡散方程式に対しては無条件安定であり，陽解法のような厳しい安定条件を受けない．そのため，大きな時間刻み幅を用いても安定した計算を行うことができるという利点を持っている．

4.1 境界条件の反映

差分法では内部格子点だけでなく、境界点における未知量も適切に扱う必要がある。本課題では、左端では Dirichlet 境界条件、右端では Neumann 境界条件を与えているため、それぞれ異なる方法で差分式へ反映した。まず、左端境界では

$$u(0, t) = 0 \quad (18)$$

が与えられている。これは Dirichlet 境界条件と呼ばれ、境界における未知量そのものが既知であることを意味する。そのため、差分近似を行う必要はなく、常に

$$u_0^n = 0 \quad (19)$$

と与えればよい。実際のプログラムにおいても、各時間ステップごとに

$$u_0^{n+1} = 0 \quad (20)$$

を代入することで境界条件を満たしている。一方、右端では

$$\frac{\partial u}{\partial x}(L, t) = 0 \quad (21)$$

が与えられている。これは Neumann 境界条件であり、境界における空間微分（流束）が零であることを意味する。この一階微分を後退差分で近似すると、

$$\frac{\partial u}{\partial x} \approx \frac{u_{N_x} - u_{N_x-1}}{\Delta x} \quad (22)$$

となる。境界条件より

$$\frac{u_{N_x} - u_{N_x-1}}{\Delta x} = 0 \quad (23)$$

であるから、

$$u_{N_x} = u_{N_x-1} \quad (24)$$

が得られる。したがって、右端境界では境界点の値を一つ内側の格子点と等しく設定することによって、Neumann 境界条件を満足させることができる。陽解法では、各時間ステップで内部格子点の更新を行った後、

$$u_{N_x}^{n+1} = u_{N_x-1}^{n+1} \quad (25)$$

と代入することで境界条件を反映した。

一方，陰解法では，この境界条件を係数行列の構築時にあらかじめ組み込んでいる．陰解法では最後の内部節点に対する差分式は

$$-ru_{N_x-2}^{n+1} + (1+2r)u_{N_x-1}^{n+1} - ru_{N_x}^{n+1} = u_{N_x-1}^n \quad (26)$$

となるが，ここで Neumann 境界条件

$$u_{N_x}^{n+1} = u_{N_x-1}^{n+1} \quad (27)$$

を代入すると，

$$-ru_{N_x-2}^{n+1} + (1+r)u_{N_x-1}^{n+1} = u_{N_x-1}^n \quad (28)$$

となる．この結果，係数行列の最終行は

$$[\cdots \quad -r \quad 1+r] \quad (29)$$

となり，本課題で作成した Fortran プログラム中の係数行列

$$A(N_x-1, N_x-2) = -r, \quad A(N_x-1, N_x-1) = 1+r \quad (30)$$

が得られる．

以上より，左端の Dirichlet 境界条件は未知量を直接固定することで，右端の Neumann 境界条件は差分近似を用いて係数行列および境界点の値へ反映することで，両境界条件を数値計算に組み込んだ．

5 解析解

数値解の精度を評価するため，拡散方程式の解析解を導出する．

対象とする支配方程式は

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial x^2} \quad (31)$$

である．境界条件は

$$u(0, t) = 0 \quad (32)$$

および

$$\frac{\partial u}{\partial x}(L, t) = 0 \quad (33)$$

であり、初期条件は

$$u(x, 0) = 1 \quad (34)$$

とする。まず、変数分離法を用いて

$$u(x, t) = X(x)T(t) \quad (35)$$

と仮定する。これを支配方程式へ代入すると、

$$X(x)\frac{dT}{dt} = \nu T(t)\frac{d^2 X}{dx^2} \quad (36)$$

となる。両辺を $\nu X(x)T(t)$ で割ると、

$$\frac{1}{\nu T} \frac{dT}{dt} = \frac{1}{X} \frac{d^2 X}{dx^2} \quad (37)$$

を得る。左辺は時間のみ、右辺は空間のみの関数であるため、両辺は同じ定数に等しくなる。ここで分離定数を $-\lambda$ とおくと、

$$\frac{1}{\nu T} \frac{dT}{dt} = \frac{1}{X} \frac{d^2 X}{dx^2} = -\lambda \quad (38)$$

となる。したがって、時間方向と空間方向の微分方程式は

$$\frac{dT}{dt} + \nu \lambda T = 0 \quad (39)$$

および

$$\frac{d^2 X}{dx^2} + \lambda X = 0 \quad (40)$$

となる。空間方向の一般解は

$$X(x) = C_1 \cos(\sqrt{\lambda}x) + C_2 \sin(\sqrt{\lambda}x) \quad (41)$$

である。左端境界条件

$$X(0) = 0 \quad (42)$$

より

$$C_1 = 0 \quad (43)$$

となるので,

$$X(x) = C_2 \sin(\sqrt{\lambda}x) \quad (44)$$

となる．次に右端境界条件

$$\frac{dX}{dx}(L) = 0 \quad (45)$$

を適用する．空間微分は

$$\frac{dX}{dx} = C_2 \sqrt{\lambda} \cos(\sqrt{\lambda}x) \quad (46)$$

となるので,

$$\cos(\sqrt{\lambda}L) = 0 \quad (47)$$

が得られる．余弦関数が零となる条件より,

$$\sqrt{\lambda}L = \frac{(2n+1)\pi}{2}, \quad n = 0, 1, 2, \dots \quad (48)$$

である．したがって,

$$\lambda = \left(\frac{(2n+1)\pi}{2L} \right)^2 \quad (49)$$

となる．ここでコード中で用いた固有値

$$B_n = \frac{(2n+1)\pi}{2L} \quad (50)$$

を導入すると,

$$\lambda = B_n^2 \quad (51)$$

と表すことができる．時間方向の微分方程式

$$\frac{dT}{dt} + \nu B_n^2 T = 0 \quad (52)$$

を解くと,

$$T(t) = \exp(-\nu B_n^2 t) \quad (53)$$

となる. したがって, 各固有モードの解は

$$u_n(x, t) = A_n \sin(B_n x) \exp(-\nu B_n^2 t) \quad (54)$$

となる. 重ね合わせの原理より, 全体の解は

$$u(x, t) = \sum_{n=0}^{\infty} A_n \sin(B_n x) \exp(-\nu B_n^2 t) \quad (55)$$

となる. 初期条件

$$u(x, 0) = 1 \quad (56)$$

を代入すると,

$$1 = \sum_{n=0}^{\infty} A_n \sin(B_n x) \quad (57)$$

となる. 両辺に $\sin(B_m x)$ を掛けて 0 から L まで積分し, 三角関数の直交性を利用すると,

$$A_n = \frac{2}{L} \int_0^L \sin(B_n x) dx \quad (58)$$

となる. 積分を実行すると,

$$A_n = \frac{4}{(2n+1)\pi} \quad (59)$$

を得る. 以上より, 本課題で用いた解析解は

$$u(x, t) = \sum_{n=0}^{\infty} \frac{4}{(2n+1)\pi} \sin(B_n x) \exp(-\nu B_n^2 t) \quad (60)$$

ただし,

$$B_n = \frac{(2n+1)\pi}{2L} \quad (61)$$

である. 実際のプログラムでは無限級数を有限項で打ち切り,

$$N_{\text{term}} = 100 \quad (62)$$

として 100 項まで計算を行った.

6 コード

これらの計算を実装した陽解法, 陰解法のコードを以下に示す.

Listing 1 陽解法プログラム

```

1 program youkaihou
2
3     use, intrinsic :: iso_fortran_env, only : real64
4     implicit none
5
6     !変数の宣言
7
8     integer :: Nx, i, n, Nterm
9
10    real(real64) :: L, nu, dt, Tmax, dx, r, t, Bn, An, error_max
11
12    real(real64), allocatable :: x(:), u_old(:), u_new(:), u_exact(:), error(:)
13
14    !具体的なパラメータの入力
15
16    print *, "区間の長さは? L "
17    read(*,*) L
18
19    print *, "拡散係数 ν は? "
20    read(*,*) nu
21
22    print *, "空間の分割数は? Nx "
23    read(*,*) Nx
24
25    print *, "時間離散化の幅は? dt "
26    read(*,*) dt
27
28    print *, "終了する時間は? Tmax "
29    read(*,*) Tmax
30
31    !空間の分割
32
33    !整数と倍精度実数で計算を行うときは, 倍精度実数に合わせてから計算する必要がある.
34    dx = L / real(Nx, real64)
35

```

```

36 allocate(x(0:Nx))
37 !これは各点の座標を表す点だから、i ループで各点に座標を挿入
38
39 do i = 0, Nx
40     x(i) = real(i, real64) * dx
41 end do
42
43 allocate(u_old(0:Nx))
44 allocate(u_new(0:Nx))
45 allocate(u_exact(0:Nx))
46 allocate(error(0:Nx))
47
48 !安定条件の判断（離散化パラメータ r を計算し陽解法が適用できるか判断）
49 r = nu * dt / dx**2
50
51 print *, "離散化パラメータはr", r
52
53 !ここで0.5を超えることがあればプログラムを終了する
54 if (r > 0.5_real64) then
55     stop "安定条件を満たしていない"
56 end if
57
58
59 !初期条件の設定（問題を参照）
60
61 u_old = 1.0_real64
62 U_old(0) = 0.0_real64
63
64 u_new = 0.0_real64
65
66 t = 0.0_real64
67
68 !離散化した時間を順番にたどり、Tmaxにおけるuを計算する
69
70 do while (t < Tmax - 1.0d-11)
71
72     !内部のノード
73
74     do i = 1, Nx-1
75
76         u_new(i) = u_old(i) + r * (u_old(i+1) - 2.0_real64*u_old(i) + u_old(
77             i-1))
78
79     end do
80
81     !境界条件
82
83     !x = 0 (Dirichlet)
84     u_new(0) = 0.0_real64

```

```

84
85     !x = L (Neumann)
86     u_new(Nx) = u_new(Nx-1)
87
88     !全ノードの情報を更新したうえで時間を更新
89     u_old = u_new
90     t = t + dt
91
92 end do
93
94
95
96 !解析解の計算
97
98 !フーリエ級数は100項もあれば十分な精度が得られるため、打ち切りとして設定
99 Nterm = 100
100
101 u_exact = 0.0_real64
102
103 do i = 0, Nx
104
105     do n = 0, Nterm-1
106
107         !式変形で得られた係数を定義
108
109         Bn = (2.0_real64*n + 1) * acos(-1.0_real64) / (2.0_real64*L)
110
111         An = 4.0_real64 / ((2.0_real64*n + 1.0_real64) * acos(-1.0_real64))
112
113         u_exact(i) = u_exact(i) + An * sin(Bn * x(i)) * exp(-nu*Bn*Bn*Tmax)
114
115     end do
116
117 end do
118
119
120 !誤差の計算
121 error_max = 0.0_real64
122
123 do i = 0, Nx
124
125     error(i) = abs(u_old(i) - u_exact(i))
126
127     if (error(i) > error_max) then
128         error_max = error(i)
129
130     end if
131
132 end do

```

```

133
134
135 !結果の出力
136 open(10, file="youkaihou.csv", status="replace")
137
138 write(10, '(a)') "座標数値解解析誤差,,,"
139
140 do i = 0, Nx
141
142     write(10, '(4(es16.8,:","))') &
143         x(i), u_old(i), u_exact(i), error(i)
144
145 end do
146
147 close(10)
148
149 print *, "Maximum error =", error_max
150
151 !メモリの解放
152
153 deallocate(x)
154 deallocate(u_old)
155 deallocate(u_new)
156 deallocate(u_exact)
157 deallocate(error)
158
159 end program youkaihou

```

Listing 2 陰解法プログラム

```

1 program inkaihou
2
3 use, intrinsic :: iso_fortran_env, only : real64
4 use mod_linear_solver_lu, only : matrix_inverse
5 implicit none
6
7 !変数の宣言
8
9 integer :: Nx, i, j, n, Nterm
10
11 real(real64) :: L, nu, dt, Tmax, dx, r, t, Bn, An, error_max
12
13 real(real64), allocatable :: x(:), u_old(:), u_new(:), u_exact(:), error(:)
14
15 real(real64), allocatable :: A(:, :), Ainv(:, :)
16
17 !具体的なパラメータの入力
18
19 print *, "区間の長さは？L"

```

```

20  read(*,*) L
21
22  print *, "拡散係数  $\nu$  は？"
23  read(*,*) nu
24
25  print *, "空間の分割数は？  $N_x$ "
26  read(*,*) Nx
27
28  print *, "時間離散化の幅は？  $\Delta t$ "
29  read(*,*) dt
30
31  print *, "終了する時間は？  $T_{max}$ "
32  read(*,*) Tmax
33
34  !空間の分割
35
36  !整数と倍精度実数で計算を行うときは、倍精度実数に合わせてから計算する必要がある。
37  dx = L / real(Nx, real64)
38
39  allocate(x(0:Nx))
40  !これは各点の座標を表す点だから、i ループで各点に座標を挿入
41
42  do i = 0, Nx
43      x(i) = real(i, real64) * dx
44  end do
45
46  allocate(u_old(0:Nx))
47  allocate(u_new(0:Nx))
48  allocate(u_exact(0:Nx))
49  allocate(error(0:Nx))
50
51  allocate(A(Nx-1, Nx-1))
52  allocate(Ainv(Nx-1, Nx-1))
53
54  !安定条件の判断（離散化パラメータ  $r$  を計算し陽解法が適用できるか判断）
55  r = nu * dt / dx**2
56
57  print *, "離散化パラメータは  $r$ ", r
58
59
60  !初期条件の設定（問題を参照）
61
62  u_old = 1.0_real64
63  U_old(0) = 0.0_real64
64
65  u_new = 0.0_real64
66
67  t = 0.0_real64
68

```

```

69      !陰解法で用いる係数行列を作成
70      A = 0.0_real64
71
72      do i = 2, Nx-2
73
74          A(i,i-1) = -r
75          A(i,i)   = 1.0_real64 + 2.0_real64*r
76          A(i,i+1) = -r
77
78      end do
79
80      A(1,1) = 1.0_real64 + 2.0_real64*r
81      A(1,2) = -r
82
83      A(Nx-1,Nx-2) = -r
84      A(Nx-1,Nx-1) = 1.0_real64 + r
85
86      ! 分解モジュールを用いて逆行列を計算LU
87      call matrix_inverse(A, Ainv)
88
89      !離散化した時間を順番にたどり、におけるを計算するTmaxu
90
91      do while (t < Tmax - 1.0d-11)
92
93          !内部のノード
94          do i = 1, Nx-1
95
96              u_new(i) = 0.0_real64
97
98              do j = 1, Nx-1
99
100                  u_new(i) = u_new(i) + Ainv(i,j) * u_old(j)
101
102              end do
103
104          end do
105
106          !境界条件
107
108          !x = 0 (Dirichlet)
109          u_new(0) = 0.0_real64
110
111          !x = L (Neumann)
112          u_new(Nx) = u_new(Nx-1)
113
114          !全ノードの情報を更新したうえで時間を更新
115          u_old = u_new
116          t = t + dt
117

```

```

118     end do
119
120
121     !解析解の計算
122
123     !フーリエ級数は100項もあれば十分高い精度が得られるため、打ち切りとして設定
124     Nterm = 100
125
126     u_exact = 0.0_real64
127
128     do i = 0, Nx
129
130         do n = 0, Nterm-1
131
132             !式変形で得られた係数を定義
133
134             Bn = (2.0_real64*n + 1) * acos(-1.0_real64) / (2.0_real64*L)
135
136             An = 4.0_real64 / ((2.0_real64*n + 1.0_real64) * acos(-1.0_real64))
137
138             u_exact(i) = u_exact(i) + An * sin(Bn * x(i)) * exp(-nu*Bn*Bn*Tmax)
139
140         end do
141
142     end do
143
144
145     !誤差の計算
146     error_max = 0.0_real64
147
148     do i = 0, Nx
149
150         error(i) = abs(u_old(i) - u_exact(i))
151
152         if (error(i) > error_max) then
153             error_max = error(i)
154
155         end if
156
157     end do
158
159
160     !結果の出力
161     open(10, file="inkaihou.csv", status="replace")
162
163     write(10, '(a)') "座標数値解析解誤差,,,"
164
165     do i = 0, Nx
166

```



```

167         write(10, '(4(es16.8, :, ", "))' ) &
168             x(i), u_old(i), u_exact(i), error(i)
169
170     end do
171
172     close(10)
173
174     print *, "Maximum error =", error_max
175
176     !メモリの解放
177
178     deallocate(x)
179     deallocate(u_old)
180     deallocate(u_new)
181     deallocate(u_exact)
182     deallocate(error)
183     deallocate(A)
184     deallocate(Ainv)
185
186 end program inkaihou

```

7 実行結果

作成したコードの実行結果を以下に示す．本課題では，メインプログラムは Fortran で実装しており，離散化パラメータと最大誤差を表示するようになっている．離散化された各格子点における数値解は csv ファイルとして出力し，そのデータの図示には，Python を使用している．

例として，具体的なパラメータを以下のように仮定して実行した結果を図として示す．

$$L = 1.0, \quad \nu = 1.0, \quad N_x = 50, \quad \Delta t = 0.0001, \quad T_{max} = 0.1$$

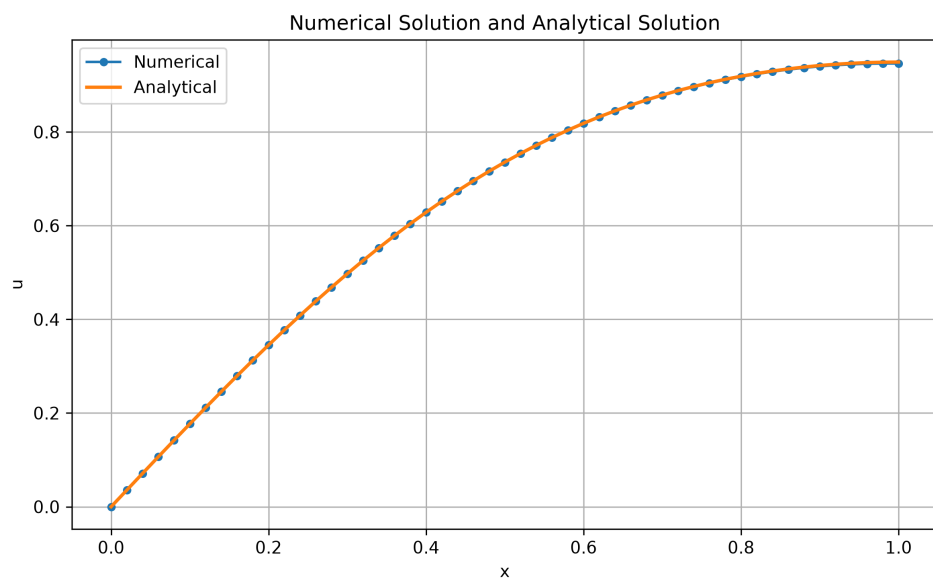


図1 陽解法の実行結果 (数値解と解析解)

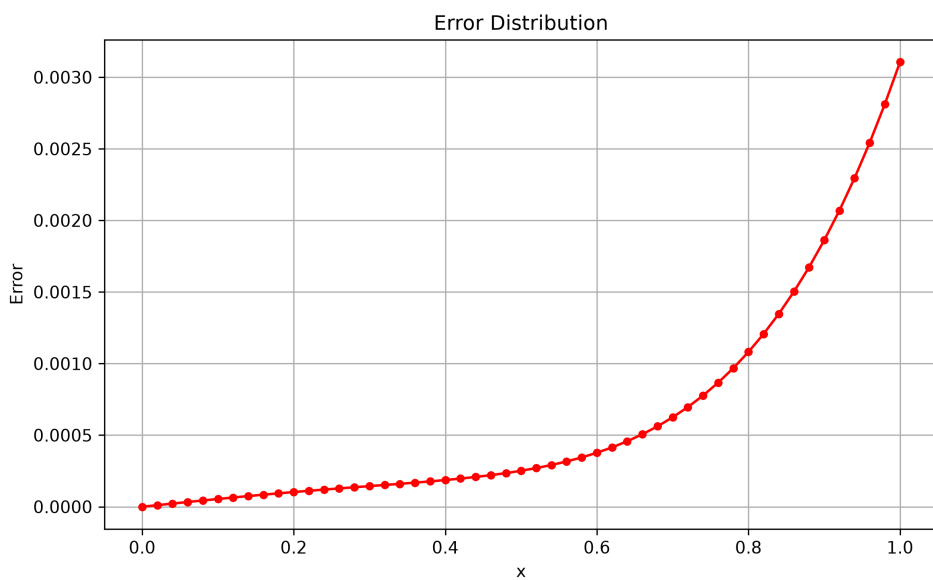


図2 陽解法の実行結果 (誤差)

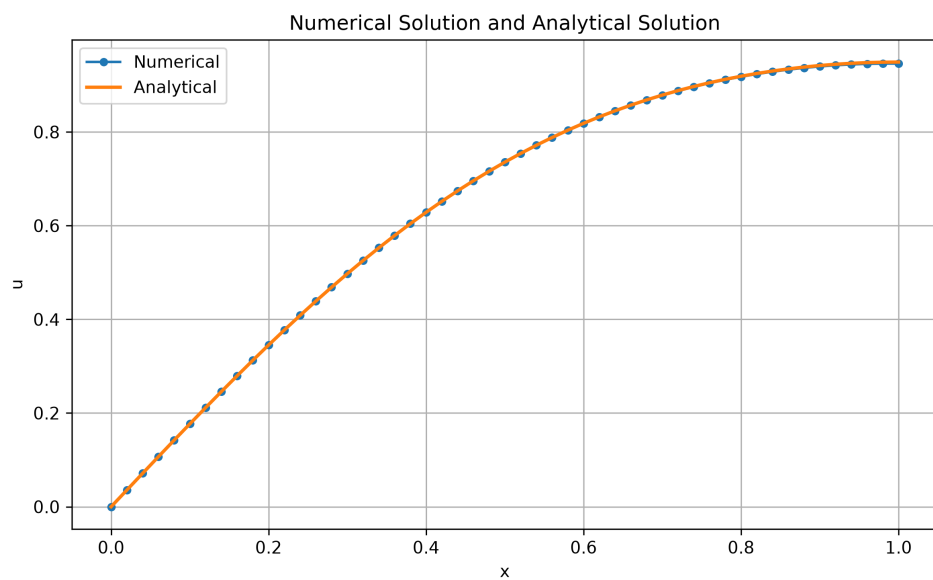


図3 陰解法の実行結果 (数値解と解析解)

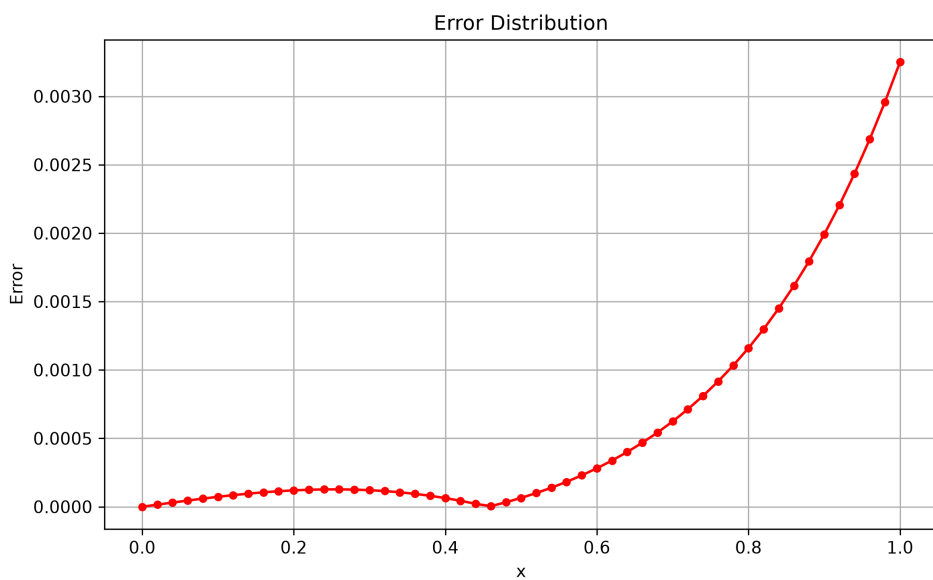


図4 陰解法の実行結果 (誤差)

ここで、陽解法における最大誤差は約 3.106×10^{-3} 、陰解法における最大誤差は約 3.250×10^{-3} であった。

8 考察

図1および図3より、陽解法、陰解法ともに数値解は解析解とほぼ重なっており、両手法とも一次元拡散方程式を適切に解くことができていることが確認できた。実際にグラフを比較しても、両者を見分けることが難しいほど一致しており、今回作成したプログラムが正しく実装できていることが分かる。また、誤差分布を示した図2および図4からも、全体として誤差は非常に小さい値に抑えられており、数値計算として十分な精度が得られていると考えられる。

最大誤差を比較すると、陽解法では約 3.106×10^{-3} 、陰解法では約 3.250×10^{-3} となり、陰解法の方がわずかに誤差が大きい結果となった。しかし、その差は約 1.4×10^{-4} 程度であり、非常に小さい値である。そのため、今回設定した計算条件では、どちらの手法を用いても解析解を十分な精度で再現できていると考えられる。

ここで、図2および図4を見ると、誤差は区間全体で一定ではなく、右端である $x = L$ に近づくにつれて大きくなっていることが分かる。この原因として、右端では Neumann 境界条件

$$\frac{\partial u}{\partial x} = 0$$

を差分近似によって

$$u_N = u_{N-1}$$

と与えていることが考えられる。内部点では左右の情報を用いる中心差分によって近似しているのに対し、境界では利用できる点が限られるため、近似精度がやや低下する。そのため、境界付近では内部点よりも離散化誤差が大きくなり、誤差が右端に集中したものと考えられる。また、拡散方程式では時間が経過するにつれて境界条件の影響が内部へ伝わっていくため、境界付近で生じた誤差が最終的な解にも反映されたと考えられる。

陽解法と陰解法を比較すると、両者は精度だけでなく計算方法にも大きな違いがある。陽解法では、現在の時刻の情報のみを用いて次の時刻を直接計算するため、アルゴリズムが単純で実装しやすく、1ステップ当たりの計算量も少ないという利点がある。しかし、

$$r = \frac{\nu \Delta t}{\Delta x^2}$$

に対して

$$r \leq 0.5$$

という安定条件を満たさなければならず、空間分割を細かくすると時間刻み幅も小さく設定する必要がある。その結果、時間ステップ数が増加し、計算時間が長くなるという欠点がある。

これに対して陰解法は、各時間ステップごとに連立一次方程式を解く必要があるため、1回の計算に要する計算量は陽解法よりも大きい。今回のプログラムでも、係数行列を作成し、LU分解を用いて逆行列を求めてから時間発展を行っているため、実際にプログラムを作成する中でも陽解法より処理内容が複雑であることを実感した。しかし、陰解法は理論的に無条件安定であるため、時間刻み幅を大きく設定しても発散しないとい

う大きな利点を持つ。そのため、計算効率よりも安定性を重視する解析や、長時間の熱伝導解析などでは陰解法の方が適していると考えられる。

今回の計算では、両手法とも解析解と良く一致し、誤差にも大きな違いは見られなかった。しかし、今回設定した時間刻み幅は陽解法の安定条件を十分満たしており、比較的細かい時間刻みで計算を行っている。そのため、陽解法でも高い精度が得られたものと考えられる。もし時間刻み幅をさらに大きくした場合には、陽解法では安定条件を満たせず発散する可能性があるのに対し、陰解法では安定して計算を続けることができる。このように、同じ拡散方程式を解く場合でも、解析条件によって適切な時間積分法を選択することが重要であることを、今回の数値計算を通して確認することができた。